

bok25

**基
础**

班、计算机组成原理



浮点数与IEEE754





1.浮点数



浮点数

1.浮点数的表示格式 1.1 浮点数的表示格式

浮点数的表示格式

$$N = (-1)^S \times M \times R^E$$

数符S：0表示正数，1表示负数，用来决定浮点数的符号

尾数M：一个二进制定点小数，一般用定点原码小数表示

阶数E：一个二进制定点整数，用移码表示

基数：隐含，一般可为2，4，16等



浮点数

1.浮点数的表示格式 1.2 浮点数的规格化

浮点数的规格化

通过调整一个非规格化浮点数的尾数和阶码的大小，使非零的浮点数在尾数的最高数位上保证是一个有效值

若基数是2，规格化数的形式应为 $\pm 0.1bb...bx2^E$ （b为0或者1）

若基数是 2^n ，规格化数的尾数部分最高n位不全为0



浮点数

1.浮点数的表示格式 1.2 浮点数的规格化

浮点数的规格化

若基数是2，规格化数的形式应为 $\pm 0.1bb\dots bx2^E$ (b为0或者1)

$$\text{正数: } 0.1bbb\dots bx2^E \left\{ \begin{array}{l} \text{最大值: } 0.11\dots 1 \\ \text{最小值: } 0.10\dots 0 \end{array} \right.$$

$$2^{-1} \leq \text{规格化正数} \leq 1 - 2^{-n}$$



浮点数

1.浮点数的表示格式 1.2 浮点数的规格化

浮点数的规格化

若基数是2，规格化数的形式应为 $\pm 0.1bb\dots bx2^E$ (b为0或者1)

负数： $1.1bbb\dots bx2^E$ { 最大值： $1.10\dots 0$
由此可见这个1是符号而不是数值
最小值： $1.11\dots 1$

$$-(1-2^{-n}) \leq \text{规格化负数} \leq -2^{-1}$$

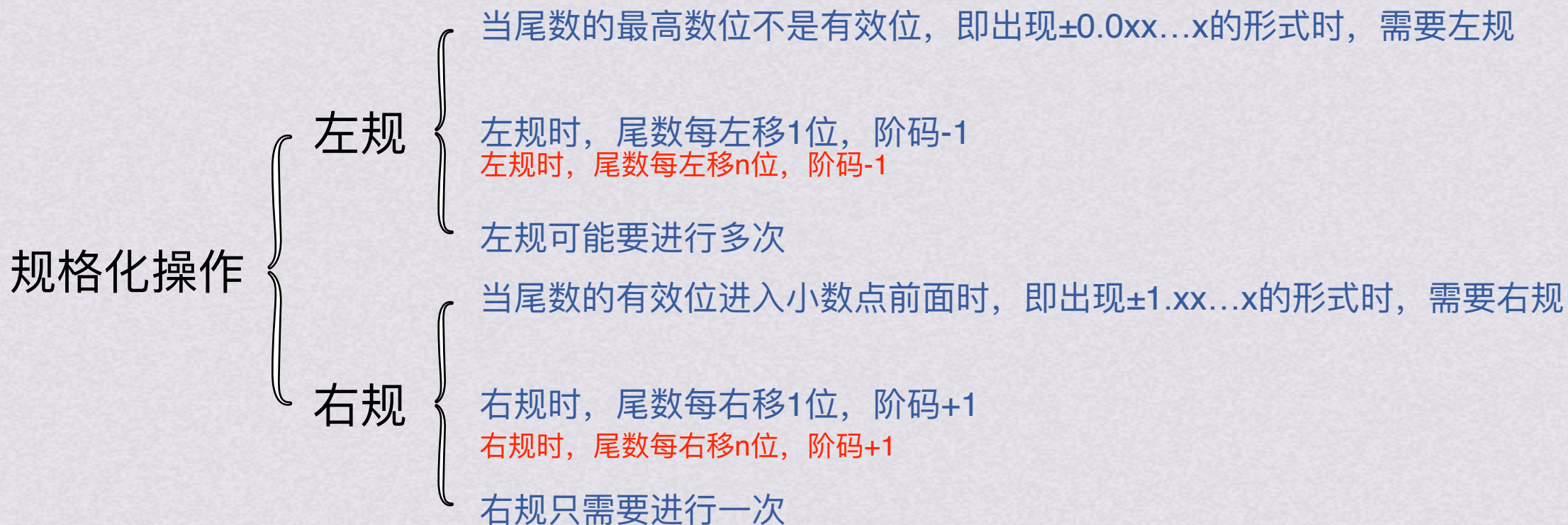


浮点数

1.浮点数的表示格式 1.2 浮点数的规格化

浮点数的规格化

以二进制浮点数为例





浮点数

1.浮点数的表示格式 1.2 浮点数的规格化

浮点数的规格化

已经学过IEEE754的同学们一定要注意区分！
这里的浮点数规格化和IEEE754的规格化不同！



你变的很陌生



浮点数

1.浮点数的表示格式 1.3 浮点数的加减运算

浮点数的加减运算

【例】阶码和尾数均采用补码表示，位数分别为5和7（均含两位符号位），
 $X=2^7 \times 29/32$ ， $Y=2^5 \times 5/8$ ，计算 $X+Y$

预处理

1.阶码：7=00111

2.尾数：29/32实际上是29右移5位

29=0011101 $\longrightarrow$ 0000000.11101



浮点数

1.浮点数的表示格式 1.3 浮点数的加减运算

浮点数的加减运算

【例】阶码和尾数均采用补码表示，位数分别为5和7（均含两位符号位），
 $X=2^7 \times 29/32$ ， $Y=2^5 \times 5/8$ ，计算 $X+Y$

预处理

$X=00\ 111; 00.11101$

$Y=00\ 101; 00.10100$



浮点数

1.浮点数的表示格式 1.3 浮点数的加减运算

浮点数的加减运算

【例】阶码和尾数均采用补码表示，位数分别为5和7（均含两位符号位），
 $X=2^7 \times 29/32$ ， $Y=2^5 \times 5/8$ ，计算 $X+Y$

对阶

先求阶差，小阶向大阶对齐，将阶码小的尾数右移，每移一位阶码加1，直到两个数的阶码相同为止

$X=00\ 111; 00.11101$
 $Y=00\ 101; 00.10100$

$\Delta=X-Y=00\ 111-00\ 101=00\ 010$ ，即Y的尾数需要右移2次



浮点数

1.浮点数的表示格式 1.3 浮点数的加减运算

浮点数的加减运算

【例】阶码和尾数均采用补码表示，位数分别为5和7（均含两位符号位），
 $X=2^7 \times 29/32$ ， $Y=2^5 \times 5/8$ ，计算 $X+Y$

对阶

先求阶差，小阶向大阶对齐，将阶码小的尾数右移，每移一位阶码加1，直到两个数的阶码相同为止

$X=00\ 111; 00.11101$

$Y=00\ 111; 00.00101$

$\Delta=X-Y=00\ 111-00\ 101=00\ 010$ ，即Y的尾数需要右移2次



浮点数

1.浮点数的表示格式 1.3 浮点数的加减运算

浮点数的加减运算

【例】阶码和尾数均采用补码表示，位数分别为5和7（均含两位符号位），
 $X=2^7 \times 29/32$ ， $Y=2^5 \times 5/8$ ，计算 $X+Y$

尾数求和

将对阶后的尾数按定点数加减运算规则进行运算

$X=00\ 111; 00.11101$

$Y=00\ 111; 00.00101$

$00.11101+00.00101=01.00010$



浮点数

1.浮点数的表示格式 1.3 浮点数的加减运算

浮点数的加减运算

【例】阶码和尾数均采用补码表示，位数分别为5和7（均含两位符号位），
 $X=2^7 \times 29/32$ ， $Y=2^5 \times 5/8$ ，计算 $X+Y$

规格化

尾数求和结束后可能会得到不符合规格化的结果，需要根据规格化格式进行尾数左规或右规
尾数右移1位，阶码加1，尾数每左移1位，阶码减1

$X=00\ 111; 00.11101$
 $Y=00\ 111; 00.00101$

00 111 01.00010

需要右规一次，尾数右移1位，阶码加1



浮点数

1.浮点数的表示格式 1.3 浮点数的加减运算

浮点数的加减运算

【例】阶码和尾数均采用补码表示，位数分别为5和7（均含两位符号位），
 $X=2^7 \times 29/32$ ， $Y=2^5 \times 5/8$ ，计算 $X+Y$

规格化

尾数求和结束后可能会得到不符合规格化的结果，需要根据规格化格式进行尾数左规或右规
尾数右移1位，阶码加1，尾数每左移1位，阶码减1

$X=00\ 111; 00.11101$

$Y=00\ 111; 00.00101$

$01\ 000\ 00.10001$

需要右规一次，尾数右移1位，阶码加1



浮点数

1.浮点数的表示格式 1.3 浮点数的加减运算

浮点数的加减运算

【例】阶码和尾数均采用补码表示，位数分别为5和7（均含两位符号位），
 $X=2^7 \times 29/32$ ， $Y=2^5 \times 5/8$ ，计算 $X+Y$

舍入

对阶和规格化过程中，可能会对尾数进行右移，造成有效位移出的情况
为了保证运算精度，一般会采取不同的舍入策略

$X=00\ 111; 00.11101$

$Y=00\ 111; 00.00101$

$01\ 000\ 00.10001$

刚才的过程中没有移出有效位，故不做舍入处理



浮点数

1.浮点数的表示格式 1.3 浮点数的加减运算

浮点数的加减运算

【例】阶码和尾数均采用补码表示，位数分别为5和7（均含两位符号位），
 $X=2^7 \times 29/32$ ， $Y=2^5 \times 5/8$ ，计算 $X+Y$

溢出判断

- 1.尾数溢出时可以通过右规进行调整，所以尾数溢出时结果不一定溢出
- 2.只要涉及到右规/左规的操作，都可能导致阶增大/减小，从而使指数发生上溢/下溢，此时判定浮点数发送溢出，产生异常/按机器零处理
- 3.浮点数的溢出并不是以尾数溢出来判断的，而是通过指数溢出来判断的

$X=00\ 111; 00.11101$

$Y=00\ 111; 00.00101$



01 000 00.10001

阶码双符号位不同，发生溢出



浮点数

1.浮点数的表示格式 1.3 浮点数的加减运算

浮点数的规格化（补码尾数）

$$\begin{array}{l} \text{正数: } 0.1\text{bbb}\dots\text{bx}2^E \left\{ \begin{array}{l} \text{最大值: } 0.11\dots1 \\ \text{最小值: } 0.10\dots0 \end{array} \right. \\ \\ \text{负数: } 1.0\text{bbb}\dots\text{bx}2^E \left\{ \begin{array}{l} \text{最大值: } 1.01\dots1 \\ \text{最小值: } 1.00\dots0 \end{array} \right. \end{array}$$

符号位与最高数值位一定相反

浮点数

1.浮点数的表示格式1.3 浮点数的加减运算

舍入

对阶和规格化过程中，可能会对尾数进行右移，造成有效位移出的情况
为了保证运算精度，一般会采取不同的舍入策略

1.0舍1入

尾数右移时，被移出的最高数位数值为0，则舍去
尾数右移时，被移出的最高数位数值为1，则在尾数末位+1

before	右移位数	after	移出位	舍入
0.0110	1	0.0011	0	0.0011
0.0111	2	0.0001	11	0.0010
0.0010	3	0.0000	010	0.0000



浮点数

1.浮点数的表示格式 1.3 浮点数的加减运算

舍入

对阶和规格化过程中，可能会对尾数进行右移，造成有效位移出的情况
为了保证运算精度，一般会采取不同的舍入策略

2.朝 $+\infty$ 方向舍入 总是取右边最近的可表示数

3.朝 $-\infty$ 方向舍入 总是取左边最近的可表示数

4.朝0方向舍入 直接截取所需位数，丢弃后面所有位



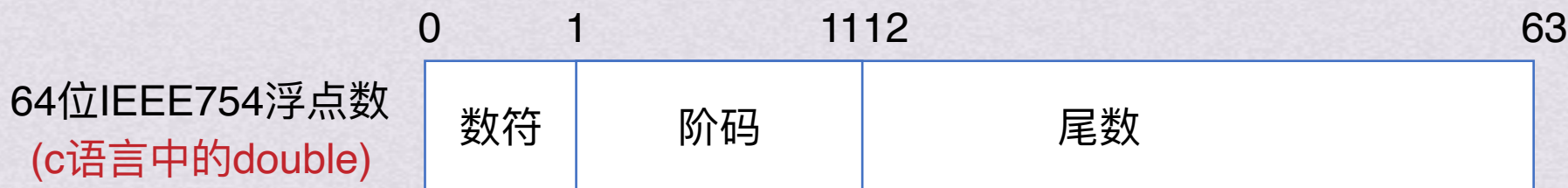
2.IEEE754



浮点数

2.IEEE754 2.1 IEEE754标准&表示格式

IEEE754标准



浮点数

2.IEEE754 2.1 IEEE754标准&表示格式

IEEE754的表示格式

类型	数符	阶码	尾数数值	总位数	偏置值
短浮点数	1	8	23	32	127
长浮点数	1	11	52	64	1023
临时浮点数	1	15	64	80	16883



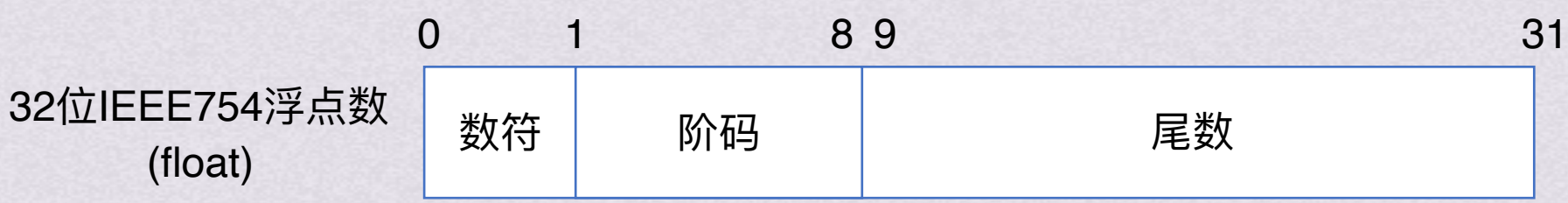
浮点数

2.IEEE754 2.2 IEEE754浮点数的规格化

IEEE754浮点数的规格化

- 对于规格化的IEEE754浮点数，数值的最高位总是1。为了能使尾数多表示一位有效位，这个1将被隐藏，称为隐藏位。因此，23位尾数实际上表示了24位有效数字。

例如，1100规格化后的结果为 1.1×2^3 （真值），其中整数部分的1将不存储在23位尾数内故该单精度浮点数的机器数为0 10000010 100.....0。



二进制表示存储数据: 0 1000 0010 100 0000 0000 0000 0000 0000

翻译为十进制数字: 0 130 翻译无意义

真实含义: 正数 (130-127=3)次方 1.1

得到结果: $1.1 \times 2^3 = 1100$

浮点数

2.IEEE754 2.3 阶码全0和全1时IEEE754浮点数的解释

阶码全0和全1时IEEE754浮点数的解释

值的类型	符号	阶码	尾数	值
正零	0	0	0	0
负零	1	0	0	0
正无穷大	0	255/2047	0	∞
负无穷大	1	255/2047	0	$-\infty$
无定义数	-	255/2047	$\neq 0$	NaN (Not a Number)
非规格化数	-	0	f	$2^{-126} \times 0.f$ $2^{-1022} \times 0.f$



浮点数

2.IEEE754

小试牛刀

【例1】 IEEE754单精度浮点格式表示的数中，最小的规格化正数是多少？



浮点数

2.IEEE754

小试牛刀

【例1】 IEEE754单精度浮点格式表示的数中，最小的规格化正数是多少？

答案： 1.0×2^{-126}



浮点数

2.IEEE754

小试牛刀

【例1】 IEEE754单精度浮点格式表示的数中，最小的规格化正数是多少？

答案： 1.0×2^{-126}

【例2】 -0.4375的IEEE754单精度浮点数表示是什么？（请用十六进制表示）



浮点数

2.IEEE754

小试牛刀

【例1】 IEEE754单精度浮点格式表示的数中，最小的规格化正数是多少？

答案： 1.0×2^{-126}

【例2】 -0.4375的IEEE754单精度浮点数表示是什么？（请用十六进制表示）

答案： BEE0 0000H



3.C语言中的浮点数类型



浮点数

3.C语言中的浮点数类型

int,float,double进行强制类型转换时，程序将得到这样的结果：

int->float，不会发生溢出，可能有数据被舍入

int/float->double，能保留精确值

double->float，可能发生溢出，可能被舍入

float/double->int，向0方向被截断